

Concepte pentru analiza si fundamentare

Algoritmi utilizati

- AutoSeg (model extern YOLO pentru segmentare + bbox)
- AutoMask (model extern de segmentare pe punct/masca; executabil dedicat)
- LLM local prin llama.cpp (llama-cpp-python)
- LLM local prin Ollama (client local pentru chat)

Protocoale utilizate

- HTTP/REST pentru autentificare si endpoint-uri CRUD
- WebSocket + STOMP pentru chat si sincronizare in timp real
- SockJS ca fallback pentru WebSocket
- AMQP (RabbitMQ) pentru evenimente intre servicii

Frameworkuri si platforme

- QT
- Python (PyQt6 pentru UI desktop)
- Java (Spring Boot pentru servicii backend)
- Docker (orchestrare servicii)

Modele folosite

- YOLO (AutoSeg pentru detectie/segmentare)
- Modele de segmentare externe (AutoMask: Mask2Former)
- LLM bazate pe transformer (llama.cpp, Ollama)

Modele abstracte

- Model de acces: utilizator, sesiune, JWT, roluri si reguli AuthN/AuthZ
- Model de colaborare: camere, prezenta, mesaje private, evenimente
- Model de stocare: Postgres pentru auth, H2/in-memory pentru chat, memorie pentru colaborare
- Model de inferenta: Transformer(pentru LLM si mask2former) si yolo

Explicatii / argumentari logice ale solutiei alese

- WebSocket + STOMP ofera timp real fara polling
- JWT securizeaza atat REST, cat si handshake-ul WebSocket
- RabbitMQ decupleaza servicii si permite scalare
- AutoSeg/AutoMask reduc timpul de adnotare

Structura logica si functionala a aplicatiei

- Client desktop (PyQt) cu unelte de adnotare

- Worker-e pentru chat LLM (llama.cpp, Ollama)
- Worker-e pentru AutoSeg/AutoMask (processe externe)
- Servicii backend: auth, chat, annotation
- Persistenta: Postgres pentru auth; in-memory pentru colaborare/chat